

Koopman-Accelerated Model-Based Diffusion for Real-Time Robot Control

Abstract—

Keywords—Koopman operator, sampling-based control, model-based diffusion, data-driven control, model learning for control.

I. INTRODUCTION

Sampling-based predictive control is widely used on robotic systems with nonlinear dynamics. MPPI [1]–[4] and Model-Based Diffusion (MBD) [5] score a population of candidate control sequences by forward rollout and update the nominal sequence by a cost-weighted average. MBD reads that update as one step of reverse diffusion. The weighted mean estimates the score of a Gaussian-smoothed target [6], [7], and a decreasing noise schedule moves the search from coarse to fine. Neither the cost nor the feasible set has to be differentiable or convex.

The rollout model keeps MBD out of the control loop. One control step evaluates the rollout once per candidate at every annealing stage, and receding-horizon control draws a new population after each executed action. MBD is posed as an offline trajectory optimizer, and its use under receding horizon is left open [5]. Rolling a population through a simulation of the plant is the dominant cost of a control step, and Xue et al. [8] later carried that cost on a vectorized physics engine, running annealed sampling at the control rate and leaving the rollout model unchanged. Reaching the control rate on a CPU requires a rollout model cheaper than a simulation of the plant.

Replacing the simulator raises a question the simulator never posed. Where must the surrogate be accurate? The sampling weights read only differences of cost within a population, and the candidates differ in the control sequence alone. An error the surrogate commits independently of the input leaves the weights unchanged. An error in the path by which the input reaches the tracked output reorders the candidates.

A Koopman lift supplies such a surrogate. Lifting the state through hand-designed [9], [10] or learned [11] observables turns a nonlinear rollout into repeated matrix–vector products whose cost the lifted dimension sets. Hao et al. [12] placed a learned linear Koopman model inside an MPPI loop for this reason. A linear lift receives the input through a constant matrix, so a command moves the tracked output in the same way at every configuration. On a velocity-commanded platform the command is issued in a joint or body frame and the tracked output lives in the world frame. The map between the two frames turns with the configuration and ranges over its full extent across the postures a task visits. A constant matrix matches it at one posture.

Bilinear Koopman realizations multiply the lifted state by the input and restore a configuration-dependent input map [13]. Lifted predictive control nonetheless builds mostly on the linear form, for which the online problem is a quadratic program [14]. Bruder et al. [15] identified a bilinear model from data, but the bilinear term breaks convexity, so they froze the lifted state along a nominal trajectory. That remedy removes the state dependence the bilinear form was adopted for. The optimizer, not the model, has kept the bilinear class out of this role. A sampling planner scores whole candidate sequences by the cost they achieve, so it requires convexity of neither the model nor the free space and uses the bilinear lift as it stands.

A learned rollout introduces a second difficulty. The optimizer may descend into a minimum that belongs to the surrogate rather than to the plant, and the robot executes whatever the sampler settles on. The annealed schedule mitigates this sensitivity rather than removing the error. The noise level sets how widely one stage spreads its candidates, and it sets how much the cost that stage reads has been smoothed. A wide level exposes the optimizer to a broader neighborhood of the current plan, while a narrow level gives the final update a less-smoothed cost and a smaller displacement. The schedule opens wide and closes narrow, separating exploration from refinement. This role is most relevant when the rollout is learned, so we compare fixed and annealed schedules under a learned rollout and repeat the schedules on an exact rollout as a control.

The diffusion in MBD is a property of the optimizer. Diffusion models used as motion priors [16]–[18] instead learn a trajectory distribution tied to the training tasks, so a new goal or cost calls for retraining. Here the rollout model is the only learned object.

We propose Bilinear Koopman Model-Based Diffusion (BK-MBD), which rolls out a bilinear deep Koopman model inside the annealed sampling loop of MBD. The model is identified from interaction data alone by multi-step training. We evaluate BK-MBD on two velocity-commanded platforms in simulation and on physical hardware, exchanging one component at a time and holding the rest fixed. The primary contributions of this study are as follows.

- **Model-Based Diffusion at the control rate.** We run annealed sampling within the control period on a CPU by lifting the measured state once per control step and propagating every candidate in the lifted space, where one rollout step is a fixed number of matrix–vector products. The full planning computation for BK-MBD, including all five annealing stages, candidate generation, lifted rollouts, cost evaluation, softmax weighting,

weighted averaging, nominal updates and extraction of u_0 , returns in 11.0 ms median and 14.7 ms worst case with no miss of a 50 ms period.

- **Annealing for learned-rollout robustness.** We show that fixed-noise sampling must trade broad exploration against low-noise refinement, while a decreasing schedule separates these roles. Under a learned rollout this separation substantially improves closed-loop accuracy and reduces sensitivity to surrogate-induced minima. The advantage disappears with exact dynamics, isolating the effect to rollout error rather than to the sampler alone.
- **What convexity costs a Koopman predictive controller.** We isolate the price of convexifying the free space by giving one trained model to two optimizers on a task no single convex region covers. A quadratic program uses a bilinear lift only after convexification, and it requires convexity of the free space as well. Holding the model fixed separates that cost from the effects of the model and of the frozen dynamics.

Section II formulates the control problem and reviews the MBD update and lifted rollouts, Section III develops BK-MBD, and Section IV presents the experiments.

II. PRELIMINARIES

In this section we formulate the control problem, review the MBD update, and state the two classes of lifted rollout that the study compares.

A. Problem Formulation

Consider the discrete-time system

$$s_{t+1} = f(s_t, u_t), \quad s_t \in \mathcal{X} \subset \mathbb{R}^n, \quad u_t \in \mathcal{U} = [\underline{u}, \bar{u}]^m, \quad (1)$$

obtained from a control period Δt . Here f is unknown or too costly to integrate at the sampling rate, so a true-dynamics rollout serves as an oracle reference alone, and the rollout model is learned from interaction data $\mathcal{D} = \{(s_i, u_i, s_i^+)\}$. Throughout, $\mathcal{X}$ is the operating set, the region of the state space to which the task confines the plant, and every statement below is made over it. The task defines features $b(s) \in \mathbb{R}^d$, which the rollout model predicts and the cost reads. The controller tracks an output $y = S_y b(s) \in \mathbb{R}^{d_y}$ selected from those features by a constant S_y of full row rank.

The plants considered here are control-affine and velocity-commanded, $\dot{s} = f_c(s) + G(s)u$ with output $y = h(s)$. Integrating over one control period under a zero-order hold gives the response of the tracked output,

$$y_{t+1} - y_t = (\omega(s_t) + \Phi(s_t) u_t) \Delta t + O(\Delta t^2), \quad (2)$$

with drift response $\omega = \nabla h f_c \in \mathbb{R}^{d_y}$ and input gain $\Phi = \nabla h G \in \mathbb{R}^{d_y \times m}$. Two features of (2) are used later. The drift response is a function of the state alone and a model of the state may absorb it, so $\Phi(s)u$ is the only term in which the state and the input meet. The input gain is in general state-dependent. How much it varies over $\mathcal{X}$ is a property of the

plant and the task rather than of the controller applied to them.

Everything the controller optimizes is produced by a *rollout model* $\mathcal{R} : (b_t, U) \mapsto (\hat{b}_1, \dots, \hat{b}_T)$ acting on a candidate sequence $U = (u_0, \dots, u_{T-1})$. The cost depends on the candidate only through $\mathcal{R}$, so two controllers that share an optimizer differ exactly in $\mathcal{R}$. At state s_t the controller solves

$$\begin{aligned} \min_U J(U) &= \sum_{k=1}^T c(\hat{b}_k, u_{k-1}) + \ell_T(\hat{b}_T) \\ \text{s.t. } U &\in \mathcal{U}^T, \quad \hat{b}_k \in \mathcal{F}, \quad k = 1, \dots, T, \end{aligned} \quad (3)$$

where $\mathcal{F}$ is the free set the task admits. It then applies u_0 to (1) and replans at the next period.

Two properties of (3) are stated as what is *not* assumed. Neither c nor ℓ_T has to be differentiable or convex in U , and $\mathcal{F}$ has to be neither convex nor connected. Since no structure is imposed on $\mathcal{F}$, an optimizer may enforce it as a constraint or absorb it into the cost as a penalty. The controller is real-time in the sense that u_0 must be returned within Δt , so the admissibility of a rollout model depends on what the optimizer asks of it in one control step.

B. The Model-Based Diffusion Update

MBD [5] treats (3) as sampling from the Boltzmann target $p_0(U) \propto \exp(-J(U)/\alpha)$ with temperature $\alpha > 0$. It descends a ladder of Gaussian-smoothed marginals $p_\sigma = p_0 * \varphi_\sigma$ over a decreasing schedule $\sigma_1 > \dots > \sigma_S$.¹ Applying Tweedie's formula to a smoothed marginal expresses its score as a denoising mean shift,

$$\nabla_U \log p_\sigma(U) = (\mathbb{E}[U_0 | U] - U) / \sigma^2. \quad (4)$$

The smoothing kernel $\varphi_\sigma(U - U')$ is the density of the proposal $U' \sim \mathcal{N}(U, \sigma^2 I)$, so the posterior mean in (4) is a ratio of expectations under that proposal. Drawing N candidates $U_k = U + \sigma \varepsilon_k$ with $\varepsilon_k \sim \mathcal{N}(0, I)$ and projecting them onto $\mathcal{U}^T$ gives its self-normalized estimator,

$$\widehat{\mathbb{E}}[U_0 | U] = \sum_{k=1}^N w_k U_k, \quad w_k = \frac{e^{-(J(U_k) - J_{\min})/\alpha}}{\sum_l e^{-(J(U_l) - J_{\min})/\alpha}}. \quad (5)$$

The update at stage j is a preconditioned score step with optional Langevin noise,

$$U \leftarrow U + \eta_j \widehat{\nabla_U \log p_{\sigma_j}}(U) + \sqrt{2\eta_j} \xi, \quad \eta_j = \eta \sigma_j^2, \quad (6)$$

At $\eta = 1$ and without Langevin noise, (6) reduces to the cost-weighted mean $U \leftarrow \sum_k w_k U_k$.

Three properties of (6) are used later, and none of them depends on the rollout model that supplies J . First, the weights (5) are invariant to any term shared by the population. Only differences of cost within a population enter the update, and the candidates of a population differ in U alone. Second, the S stages are serial, since stage j samples around the iterate produced by stage $j - 1$. The

¹We write the ladder in variance-exploding form, in which the smoothing kernel alone carries the schedule. The formulation of [5] additionally rescales the iterate, which affects no statement below.

N candidates within a stage and the T steps of a rollout admit batching. Third, a single stage at a fixed noise level, with $\eta = 1$ and no Langevin noise, is one cost-weighted average over a Gaussian population drawn around the warm-started iterate. That average is the MPPI update [3], so the annealed schedule is the property separating (6) from the planner it generalizes, and a comparison at a matched total sample budget is available by construction.

These properties fix the cost of one control step. The rollout model is evaluated once per candidate at every stage and over the full horizon, so a control step issues NST queries to $\mathcal{R}$. Among them the S stages lie on a serial chain that added parallel hardware cannot shorten.

C. Linear and Bilinear Lifted Rollouts

The Koopman operator [19], [20] advances observables of the state along the flow and is linear on the space of observables, but it is infinite-dimensional. For a system driven by an input, an exact realization requires observables of the input as well as of the state [13]. A finite-dimensional realization therefore truncates the representation of the state and, separately, fixes the form in which the input enters the lifted dynamics. The two classes below agree in the first respect and differ in the second.

We use a learned lift with an exact-decoder convention, $z = \Psi_\theta(b) \in \mathbb{R}^r$, whose leading d coordinates hold b itself and whose remainder is learned. Placing the features in the leading coordinates makes the decoder C with $Cz = b$ a constant selector, and composing it with the output map gives the constant readout $C_y = S_y C \in \mathbb{R}^{d_y \times r}$. Two classes of lifted dynamics are available,

$$\text{linear: } z^+ = Az + Bu, \quad (7)$$

$$\text{bilinear: } z^+ = Az + B_0 u + \sum_{i=1}^m u_i B_i z, \quad (8)$$

the latter written compactly as $z^+ = M(u)z + B_0 u$ with $M(u) = A + \sum_i u_i B_i$. For control-affine systems the continuous-time Koopman generator carries a bilinear input structure [13], of which (8) is the finite-dimensional, discrete-time counterpart. Realizations of this form have been identified from data and used for robot control [15].

The two classes differ in exactly one place, the input tangent map. For (8), $\partial z^+ / \partial u_i = B_0 e_i + B_i z$ depends on the state through $B_i z$. For (7), $\partial z^+ / \partial u = B$ is one matrix for the whole of $\mathcal{X}$. Both remain in the same computational regime, a bilinear step costing $m+1$ matrix-vector products of size r against one for the linear form. The cost of a rollout step is therefore set by the lifted dimension and the number of input channels alone.

III. BILINEAR KOOPMAN MODEL-BASED DIFFUSION

BK-MBD is the optimizer of Sec. II-B with every rollout performed by a lifted surrogate of Sec. II-C. Sec. III-A fixes the class of that surrogate and Sec. III-B identifies a model of that class from data. The rollout is then a learned object rather than a simulator, and Sec. III-C treats what the noise schedule has to do as a result. Throughout this section $\|\cdot\|$ denotes the induced 2-norm.

A. The Bilinear Rollout

Sec. II-C leaves the learned coordinates of the lift unspecified. We take

$$z = \Psi_\theta(b) = \begin{bmatrix} b \\ \psi_\theta(\tau(b)) \end{bmatrix} \in \mathbb{R}^r, \quad (9)$$

where ψ_θ is a small neural network and τ is a fixed task input map that supplies features the learned part would otherwise have to construct. We write Ψ_i for the i th component of Ψ_θ .

The weights (5) read only differences of cost within a population whose members differ in the control sequence alone. What governs the optimizer is therefore the response the rollout predicts to an applied input. Decoding one step of the linear class (7) gives $C_y z^+ = C_y A z + C_y B u$, so that response has sensitivity $C_y B$, a single matrix over the whole of $\mathcal{X}$. The lifted dimension r and the encoder Ψ_θ change the value of $C_y B$ and leave it a single matrix, since C_y is a constant selector and B carries no dependence on the state. The plant answers with $\Delta t \Phi(s)$ by (2), which takes a different value at every configuration.

Lemma 1 (Irreducible input-channel error of a linear lift [21]): Let $\nu_\Phi = \sup_{s, s' \in \mathcal{X}} \|\Phi(s) - \Phi(s')\|$ denote the variation of the input gain over the operating set. For every constant matrix $M \in \mathbb{R}^{d_y \times m}$,

$$\sup_{s \in \mathcal{X}} \|\Delta t \Phi(s) - M\| \geq \frac{1}{2} \nu_\Phi \Delta t. \quad (10)$$

In particular the bound holds at $M = C_y B$ for every pair (A, B) of (7), and therefore at every lifted dimension and for every encoder.

Proof: For any $s, s' \in \mathcal{X}$ the triangle inequality gives

$$\|\Delta t \Phi(s) - M\| + \|\Delta t \Phi(s') - M\| \geq \Delta t \|\Phi(s) - \Phi(s')\|,$$

so the larger of the two terms is at least $\frac{\Delta t}{2} \|\Phi(s) - \Phi(s')\|$. Taking the supremum over pairs (s, s') yields (10). The decoded input sensitivity of (7) is one such constant matrix at every lifted dimension and for every encoder, which gives the final claim. ■

The bound depends on the variation of the plant gain alone. Nor is the deficit confined to the first step, since $\partial(C_y z_k) / \partial u_j = C_y A^{k-j-1} B$ depends on the elapsed steps $k-j$ and not on the state at which the input is applied. A linear rollout therefore predicts the same response to a given input at every configuration a candidate visits.

The bilinear class removes the bound. Its decoded input sensitivity

$$\frac{\partial(C_y z^+)}{\partial u_i} = C_y b_{0,i} + C_y B_i z, \quad i = 1, \dots, m, \quad (11)$$

is affine in the lifted state, with $b_{0,i}$ the i th column of B_0 . The predicted response then matches (2) to first order at every configuration whenever

$$C_y b_{0,i} + C_y B_i \Psi_\theta(b(s)) = \Delta t \Phi_i(s), \quad s \in \mathcal{X}, \quad (12)$$

where Φ_i is the i th column of Φ . Since $C_y = S_y C$ composes two selectors of full row rank, (12) is satisfiable whenever the entries of Φ lie in the span of $\{1, \Psi_1, \dots, \Psi_r\}$ over $\mathcal{X}$.

Any class whose input sensitivity may depend on the state satisfies (12) as well, an unstructured network among them. What (8) adds is that the dependence enters in the affine form that (2) exhibits.

On a velocity-commanded platform the map from the command frame to the world frame turns as the platform reconfigures. It ranges over its full extent across the postures a task visits, so ν_Φ stays bounded away from zero and (10) bars the linear class. Such a map is built from sines and cosines of the configuration angles and is multilinear in those terms. Letting τ supply those trigonometric functions places the leading terms in the lift and leaves ψ_θ to carry the products among them, which is the form (12) asks for. On a plant whose gain varies little the bound is small, and [21] turns the variation of Φ into a test on a given operating set.

The cost of the resulting rollout follows from the count of Sec. II-B. One control step lifts the measured features once and propagates every candidate in the lifted space afterwards, so the encoder is evaluated once per control step rather than once per candidate. Each propagation step costs $m + 1$ matrix–vector products of size r . The S stages run in series, so the control period is divided among S batched rollouts of N candidates over T steps.

B. Identification

The lifted features and the matrices of the lifted dynamics are fitted jointly to recorded trajectories. Koopman models are ordinarily identified by a one-step fit [11], which suits a model queried one step at a time. A sampling planner instead propagates a candidate over the whole horizon and separates candidates by the cost that propagation accumulates. A model that is accurate at one step and drifts over T has its drift read as a property of the plant. We therefore set H to the planning horizon and fit every model on length- H snippets by

$$\mathcal{L} = \frac{1}{H} \sum_{h=1}^H \left[\|C\hat{z}_h - b_h\|^2 + \gamma \|\hat{z}_h - \Psi_\theta(b_h)^{\text{sg}}\|^2 \right], \quad (13)$$

starting the rollout at $\hat{z}_0 = \Psi_\theta(b_0)$ and proceeding by (7) or (8). The first term measures the prediction error of the decoded features, which is the quantity the cost reads. The second term, weighted by $\gamma > 0$, holds the propagated latent state to the image of the encoder, which (7) and (8) leave it free to leave. The stop-gradient on the latent target prevents the encoder from lowering the loss by pulling the target toward the prediction.

The bilinear input matrices are initialized at $B_i = 0$. Training then begins exactly at the linear model (7), and the coupling term departs from zero only to the extent that the data support it. The two classes share a starting point, an encoder, a dataset and an objective, and they differ in whether the coupling term is permitted to move.

C. Annealing under a Learned Rollout

A learned rollout evaluates $\hat{J} = J + \Delta J$, where ΔJ collects the effect of model error. The weights (5) are invariant to the part of ΔJ common to the population, and

the remaining part reorders candidates. A learned rollout therefore admits minima of its own, and under receding horizon the robot executes whatever the sampler settles on. The schedule cannot remove this error from the rollout. Its role is instead to reduce how strongly one control step depends on a single local view of the surrogate cost.

Two properties of the update (6) bear on this, and both belong to the sampling rather than to the model. The first is how widely one stage samples. The proposals $U_k = U + \sigma \varepsilon_k$ are spread about the current plan at a scale set by σ , so σ fixes how far from that plan the cost is evaluated at all. The same scale bounds the update, which is a convex combination of those proposals. A Gaussian tail bound followed by a union bound gives

$$|\langle U^+ - U, v \rangle| \leq \sigma \sqrt{2 \log(N/\rho)} \quad (14)$$

along any unit direction v with probability at least $1 - \rho$. The width a stage explores is therefore set primarily by the noise level. Inequality (14) is an upper limit rather than a description, and a stage whose population holds nothing better than the current plan returns a weighted mean close to that plan at any σ .

The second is what one stage does with that width. The update moves the nominal by $\sigma \sum_k w_k \varepsilon_k$, whose per-axis size is

$$|\langle U^+ - U, v \rangle| \sim \sigma / \sqrt{N_{\text{eff}}}, \quad N_{\text{eff}} = 1 / \sum_k w_k^2, \quad (15)$$

so the noise level sets how large a step the stage takes. It also sets where the step points, since by (4) the stage ascends $\log p_\sigma$ and p_σ is the target smoothed at that level. A stage therefore descends the cost smoothed at its own noise level.

One noise level fixes both quantities, so a fixed-noise sampler must trade exploration against refinement. A small σ gives local proposals, a less-smoothed cost and a small final displacement, but it can remain tied to a surrogate-induced local minimum. A large σ evaluates a broader neighborhood, but it reads a more heavily smoothed cost and produces a coarser update. This does not establish that a larger noise level is better; it only characterizes the scale at which the stage reads and moves. The empirical question is whether combining scales is preferable to assigning the same sample budget to one scale.

The annealed schedule separates the two roles. It opens at a wide level, where the population is spread furthest from the current plan, and closes at a narrow level, where the smoothing and the final displacement are smallest. Under receding horizon the executed command comes from the last stage. The early stages therefore expose the optimizer to a broader neighborhood of the current plan, while the last stage refines the executed command under a less-smoothed surrogate cost. In the single-basin reaching task below, this predicts that the schedule’s advantage should be tied to learned-rollout error, a point tested by repeating the schedules with an exact rollout.

IV. EXPERIMENTS

In this section we evaluate BK-MBD on two velocity-commanded platforms in simulation and on a physical manipulator. Sec. II-A defines a controller as the pair of an

TABLE I: Hyperparameters.

	Manipulator	Drone
Lift r	20	13
Input map τ	$[\sin q, \cos q]$	identity
Planner N, S	800, 5	800, 5
Noise σ	$1.2 \rightarrow 0.3$	$1.6 \rightarrow 0.3$
Horizon T , temperature α	15, 0.4	25, 0.5

$\Delta t = 50$ ms, tolerances 0.05 and 0.01 m, $\eta = 1$ and no Langevin noise. Computed on an Intel Core Ultra 5 225F with ten cores and no GPU.

optimizer and a rollout model, so every comparison below holds one of the two fixed and exchanges the other. Sec. IV-A states the platforms and the protocol, and the studies that follow exchange the rollout class, the noise schedule and the optimizer in turn.

A. Setup

Platforms. Sec. III-A leaves two quantities to be instantiated on each platform, the map through which a command reaches the tracked output and the fixed input map τ of (9). The Franka Research 3 (FR3) accepts joint velocities through a gravity-compensated servo, tracks the tool center point (TCP) between its fingertips, and is observed as $b = [q, p_{\text{tcp}}]$. Its map is the manipulator Jacobian, whose entries are multilinear in $\{\sin q_i, \cos q_i\}$ across joints, so we take $\tau(b) = [\sin q, \cos q]$. The drone accepts body-frame linear velocities and a yaw rate, tracks its world-frame position, and is observed as $b = [p, \sin \psi, \cos \psi]$. Its map is the yaw rotation, whose entries the observation already carries, so τ is the identity. The manipulator carries the studies of Secs. IV-B and IV-C and the hardware study of Sec. ??, since DK-MBD-SPLIT requires forward kinematics in closed form, and the drone carries the non-convex passage of Sec. IV-D.

Task and protocol. Each trial starts from rest and drives the tracked output toward a goal over a fixed budget of 120 control steps, without early termination. A trial reaches a tolerance at the first control step whose tracking error falls below it, and we report the 5 cm and 1 cm tolerances. Where a condition fails to reach, we report the tracking error at the end of the budget. All conditions share one implementation of the optimizer of Sec. II-B, which draws $N = 800$ candidates over $S = 5$ stages, warm-starts each step from the previous plan by a one-step shift, and scores candidates by the squared tracking error with a control penalty and a terminal penalty. Models follow Sec. III-B on 6000 snippets of length 15 on the manipulator and 4000 on the drone, with one dataset per training seed. The closed-loop studies cross 5 training seeds with 10 goals for 50 trials per condition, and the trials are paired across conditions. Table I collects the settings that differ by platform.

Compared rollout models. Six rollout models are compared inside the identical planner, and the five learned ones form a ladder graded by how the input reaches the tracked output. MBD-TRUE rolls out the true dynamics as the oracle reference. DK-MBD rolls out the linear model (7), whose decoded input sensitivity is the constant matrix that Lemma 1 bounds. DK-MBD-LARGE raises the lifted dimension above

TABLE II: Reaching accuracy under an identical planner.

Rollout model	Reach 5 cm	Reach 1 cm
MBD-TRUE ^a	10/10	10/10
DK-MBD	10/50	1/50
DK-MBD-LARGE	11/50	1/50
MLP-MBD	41/50	0/50
DK-MBD-SPLIT ^b	50/50	50/50
BK-MBD (ours)	50/50	50/50

^a The oracle carries no learned model, so it is run over the 10 goals alone, while the learned rows span 5 seeds $\times$ 10 goals.

^b Diagnostic condition supplied with the analytic forward kinematics.

that of the bilinear model, which adds parameters and leaves that sensitivity constant. MLP-MBD is an unstructured network trained on the same data under the same loss, and its input sensitivity depends on the state. DK-MBD-SPLIT applies the analytic forward kinematics to the decoded joints, so the input reaches the tracked output outside the learned part of the model. It is a diagnostic rather than a controller, since Sec. II-A does not assume the kinematics of the plant. BK-MBD represents the same path inside the model and asks for no analytic kinematics.

B. Rollout Class and Planning Latency

This study exchanges the rollout model inside one planner and leaves everything else fixed. Table II reports reaching accuracy on the manipulator. The linear rollout reached the 1 cm tolerance on 1/50 trials and came to rest a median 0.248 m from the goal. The bilinear rollout reached it on 50/50 and matched the 10/10 of the oracle. DK-MBD-SPLIT also reached 50/50, and it differs from DK-MBD in the input sensitivity alone, so Lemma 1 accounts for the failure of DK-MBD. Fig. 1 draws one goal of that comparison.

Two further conditions exclude the remaining explanations. Raising the linear lift from $r = 20$ to $r = 50$ gave the linear model more parameters and more rollout arithmetic than the bilinear model, and it still reached 1/50 and stopped a median 0.239 m away. The bound (10) contains no lifted dimension, so a larger dictionary leaves the input sensitivity a single matrix. The unstructured network reached 41/50 within 5 cm and 0/50 within 1 cm, stopping a median 0.054 m away. Its input sensitivity depends on the state, so it can represent the input channel, and it did not identify that channel from the same data under the same objective. Capacity and expressiveness therefore leave the failure in place, and the input channel accounts for it.

Table III reports the end-to-end planning time per receding-horizon control step. The timing includes all five annealing stages, candidate generation, rollout, cost evaluation, softmax weighting, weighted averaging, nominal-sequence updates and extraction of the executed command u_0 . The oracle stepped the simulator at a median of 2598 ms and overran the 50 ms period at every one of its 1200 control steps. The BK-MBD planner returned at a median of 11.0 ms with a worst case of 14.7 ms and missed no deadline over all 50 trials. The oracle is batched over four times the threads

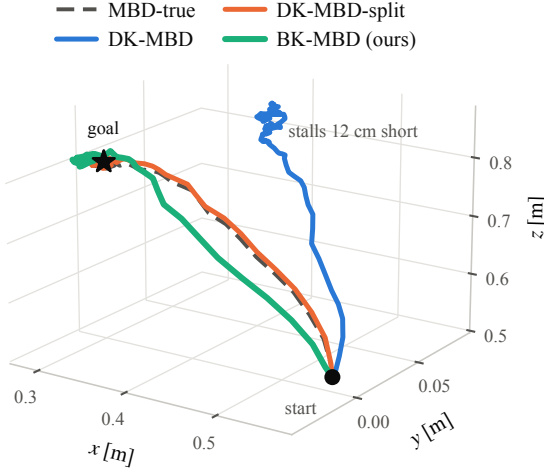


Fig. 1: Executed tool paths on one goal, one of the trials Table II counts. The four runs share the planner, the training seed and the random stream, so they differ in the rollout model alone. DK-MBD climbs away from the goal and stalls 0.117 m from it, while DK-MBD-SPLIT (0.009 m), BK-MBD (0.016 m) and the oracle MBD-TRUE (0.007 m) reach the 1 cm band.

TABLE III: End-to-end planning latency per receding-horizon control step over all trials.

Rollout model	Median [ms]	Worst [ms]	Misses
MBD-TRUE	2598	2745	1200
DK-MBD	7.6	12.5	0
BK-MBD (ours)	11.0	14.7	0

Timings include all planner computations through the selected u_0 ; misses count control steps whose planning exceeded the 50 ms period.

of the learned rollouts, so the count of Sec. II-B accounts for the gap. One control step asks the rollout model for $NST = 60,000$ steps, which the 2 ms integrator turns into 1.5 million physics substeps, and the $ST = 75$ batched steps among them lie on a serial chain. The five stages run in series, so the period admits each of them within a 10 ms share.

The bilinear rollout cost 11.0 ms against 7.6 ms for the linear one, so the configuration-dependent input map is paid for well below the $m + 1$ products a bilinear step issues, which share the lifted state they multiply.

C. The Contribution of the Annealed Schedule

This study holds the rollout class at the bilinear model and takes the noise schedule as the free variable. Five schedules are compared at one total budget of NS candidates per control step and at the temperature of Table I, a single wide stage, a single narrow stage, S wide stages, S narrow stages, and the annealed schedule from wide to narrow. The wide and narrow levels are the endpoints of the schedule in Table I, $\sigma = 1.2$ and $\sigma = 0.3$. The single-stage conditions draw NS candidates at once, which by the third property of Sec. II-B is the MPPI update at the same budget and raises the sample

TABLE IV: Schedules at an equal total sample budget, tolerance 1 cm. The lower block repeats two of them on a rollout that carries no model error.

Schedule	Reach 1 cm	Final ^a
<i>Learned bilinear rollout</i>		
$1 \times NS$ wide (MPPI) ^b	33/50	0.010
$1 \times NS$ narrow (MPPI) ^b	15/50	0.041
$S \times N$ wide	46/50	0.012
$S \times N$ narrow	26/50	0.045
$S \times N$ anneal (ours)	50/50	0.0097
<i>True-dynamics rollout^c</i>		
$S \times N$ narrow	10/10	0.009
$S \times N$ anneal	10/10	0.007

^a Median tracking error [m] at the end of the budget, which unlike the counts is not bounded by the tolerance.

^b A single stage at a fixed noise level with $\eta = 1$ and no Langevin noise is the MPPI update at the same total sample budget.

^c The oracle carries no learned model, so it is run over the 10 goals alone, on the planner stream of the block above.

count S -fold within one stage. The tolerance is set to 1 cm, where the schedules separate, and they run on identical seeds, goals and planner streams, so every comparison below is paired. The reaching task places no obstacle between the start and the goal, so the true cost supplies a control case in which the schedule should not need to overcome a multimodal true landscape.

Table IV reports the outcome. The annealed schedule reached the 1 cm band on all 50 trials and left the lowest tracking error at the end of the budget. The first question is whether annealing improves on a fixed noise level at the same total sample budget. Against the single-stage wide and narrow MPPI conditions, the annealed schedule gains 17 and 35 goals and loses none. The second question is whether the gain is merely the result of applying several serial updates. Against S repeated wide stages and S repeated narrow stages, annealing gains 4 and 24 goals. Multiple stages alone therefore do not account for the result; the changing noise scale is the separating factor.

The third question is whether the gain is tied to learned-rollout error. Rolling the settled plan of each schedule through the model and through the true dynamics locates where each one stops. Under the narrow schedule the model puts the terminal error of that plan at 0.024 m while the plant delivers 0.053 m. The plan therefore looks locally favorable under the surrogate but substantially worse under the plant. The annealed schedule returns 0.020 m under the model against 0.015 m under the plant, so its settled plan lies in a region where the two evaluations agree more closely.

We then repeated two schedules on the true-dynamics rollout, over the same goals and the same planner stream. The schedules stop separating in success rate. The narrow schedule reaches the 1 cm band on 10/10 goals as the annealed schedule does, against 26/50 under the learned rollout, and its settled error moves from 0.045 to 0.009 m. The narrow level settles quickly on either rollout, so what it loses under the learned one is not an inability of the sampler to refine a plan. It is sensitivity to the surrogate landscape.

TABLE V: Step size and step quality at a fixed noise level.

σ	$ \Delta U $	Error removed	Efficiency ^a
0.3	0.109	0.00665	61.7
0.8	0.292	0.00571	19.2
1.2	0.418	0.00409	9.9
anneal	0.109	0.00525	48.4

^a Error removed per unit of control displacement, scaled by 10^3 . $|\Delta U|$ is the displacement of the plan over one control step and the error removed is the reduction in tracking error over that step, both averaged over control steps and trials.

The wide level pays at the other end. Table V reports the control displacement one control step produces and the tracking error it removes. The displacement grows by a factor of 3.8 from $\sigma = 0.3$ to $\sigma = 1.2$ while the error removed falls, so the error removed per unit of control drops by a factor of six. This trend is consistent with the interpretation of Sec. III-C: larger noise produces broader and coarser updates, and stronger smoothing makes them less suited to final refinement. The wide schedules therefore come to rest at the edge of the band without settling inside it. No fixed level therefore meets both conditions at once. The annealed schedule executes at the displacement of the narrow level, 0.109, retains an efficiency of 48.4 against the 9.9 of the wide level, and reaches on all 50 goals. Its displacement coincides with the narrow level because the final stage produces the executed step, while the preceding wider stages have already exposed the iterate to a broader neighborhood of the current plan.

D. Non-Convex Passage

This study holds the rollout model fixed and exchanges the optimizer. A wall carries a single circular aperture, the start lies on the near side, and the goals are sampled on the far side. The straight chord from the start to any goal crosses the wall below the aperture, so every reaching motion must route through the opening (Fig. 2). The drone flies this task over 50 trials, and at each trial the two planners receive one trained bilinear model. BK-MBD absorbs the wall into the cost as a penalty, which Sec. II-A leaves to the optimizer. The convexified controller instead relinearizes the keep-out set at every step and solves the resulting quadratic program over four sequential iterations along the nominal bilinear rollout [15].

BK-MBD reached the 1 cm tolerance on 50/50 goals and the convexified controller on 10/50, and both executed zero constraint violations. Every failure of either planner was a stall at a collision-free pose, so on identical goals the two separated on passage completion at equal safety.

The structure of the free space accounts for the convexified controller. That space is the union of the two sides of the wall joined through the aperture, and a convex program retains one region containing the current nominal. Any such region that excludes the wall lies entirely on the near side. The far-side goal therefore falls outside the feasible set at every step, and the solve converges to the point of its own region closest to the goal. Two measurements match that account. The convexified controller settled a median of 0.030 m from the

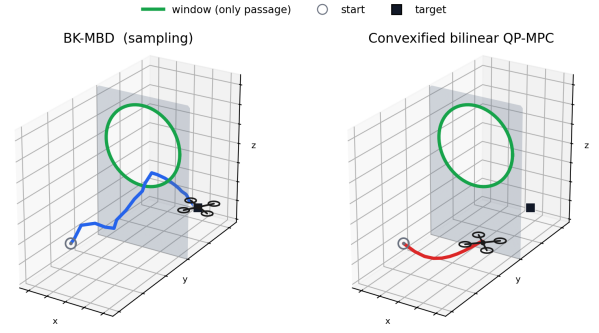


Fig. 2: Non-convex passage on the drone. The wall admits a single circular aperture. BK-MBD (left) threads the aperture and reaches the far-side goal, while the convexified bilinear controller (right) holds the near side.

wall, which is the planning margin of its own keep-out set. The ten goals it did reach were the ten whose offset from the aperture axis was smallest, at most 0.148 m against a median of 0.291 m over all goals. BK-MBD scores whole candidate sequences by the cost they achieve, so a candidate routed through the aperture receives its weight from the outcome it produces.

Two convexifications act on that controller at once, one of the dynamics and one of the free space. The two measurements above bear on the second. A controller limited by the frozen dynamics would fail by tracking a poor nominal, whereas this one stalls at the margin of its own keep-out set and succeeds on exactly the goals nearest the aperture axis. We therefore attribute the separation to the convexification of the free space, and remedies that partition that space into several convex regions remain open.

[TODO: Hardware study (subsec:exp_hw) to follow.]

V. CONCLUSION

[TODO: To be written last.]

REFERENCES

- [1] H. J. Kappen, “Path integrals and symmetry breaking for optimal control theory,” *J. Stat. Mech.: Theory Exp.*, p. P11011, 2005.
- [2] E. A. Theodorou, J. Buchli, and S. Schaal, “A generalized path integral control approach to reinforcement learning,” *J. Mach. Learn. Res.*, vol. 11, pp. 3137–3181, 2010.
- [3] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, “Information theoretic MPC for model-based reinforcement learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 1714–1721.
- [4] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Aggressive driving with model predictive path integral control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2016, pp. 1433–1440.
- [5] C. Pan, Z. Yi, G. Shi, and G. Qu, “Model-based diffusion for trajectory optimization,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, pp. 57914–57943, 2024.
- [6] Y. Song and S. Ermon, “Generative modeling by estimating gradients of the data distribution,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [7] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *International Conference on Learning Representations (ICLR)*, 2021.

- [8] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, "Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4974–4981.
- [9] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition," *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [10] J. L. Proctor, S. L. Brunton, and J. N. Kutz, "Dynamic mode decomposition with control," *SIAM J. Appl. Dyn. Syst.*, vol. 15, no. 1, pp. 142–161, 2016.
- [11] B. Lusch, J. N. Kutz, and S. L. Brunton, "Deep learning for universal linear embeddings of nonlinear dynamics," *Nat. Commun.*, vol. 9, p. 4950, 2018.
- [12] W. Hao, Y. Fang, Z. Lu, and S. Mou, "Accelerating sampling-based control via learned linear Koopman dynamics," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2026.
- [13] L. C. Iacob, R. Tóth, and M. Schoukens, "Koopman form of nonlinear systems with inputs," *Automatica*, vol. 162, p. 111525, 2024.
- [14] M. Korda and I. Mezić, "Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control," *Automatica*, vol. 93, pp. 149–160, 2018.
- [15] D. Bruder, X. Fu, and R. Vasudevan, "Advantages of bilinear Koopman realizations for the modeling and control of systems with unknown dynamics," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4369–4376, 2021.
- [16] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *International Conference on Machine Learning (ICML)*, 2022.
- [17] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.
- [18] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, "Motion planning diffusion: Learning and planning of robot motions with diffusion models," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1916–1923.
- [19] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. Natl. Acad. Sci.*, vol. 17, no. 5, pp. 315–318, 1931.
- [20] I. Mezić, "Spectral properties of dynamical systems, model reduction and decompositions," *Nonlinear Dyn.*, vol. 41, pp. 309–325, 2005.
- [21] K. Lee and S. Kim, "Linear or bilinear: A criterion for Koopman rollouts in sampling-based predictive control," *Int. J. Control Autom. Syst.*, 2026, submitted.